Warps and Thread Divergence
Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

March 2, 2025



Instructor: Thomas Feys

1 Introduction

In this lab, we explore GPU optimization techniques using CUDA. Exercise one involves implementing an image inversion kernel (x = 255 - x for each RGB component) while analyzing the impact of thread and warp configurations. Striding is introduced to handle fewer threads than image pixels efficiently.

Exercise 2 extends the kernel by applying a 1D Gaussian blur to the red channel while inverting green and blue. This introduces thread divergence, which we analyze by measuring execution time and comparing different memory layouts.

2 Solution

2.1 Warps

In the implementation with just a warp (without striding), we had to divide the picture into blocks of data corresponding to the number of threads. Striding makes it easier to handle larger datasets because a single thread can process multiple pieces of data at once, allowing for more efficient utilization of available resources.

In this implementation that makes use of striding, we observe that execution time decreases as more threads are used. This is logical, as more threads allow for more computations to happen in parallel. We see diminishing returns around the 128-thread point. Beyond this, increasing the number of threads does not significantly improve performance.

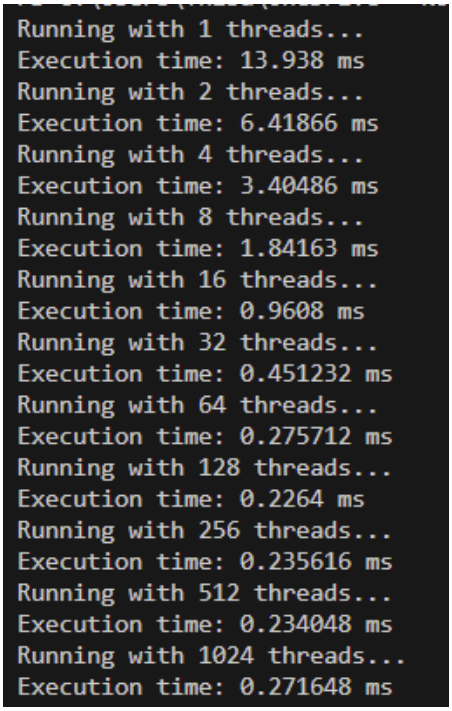
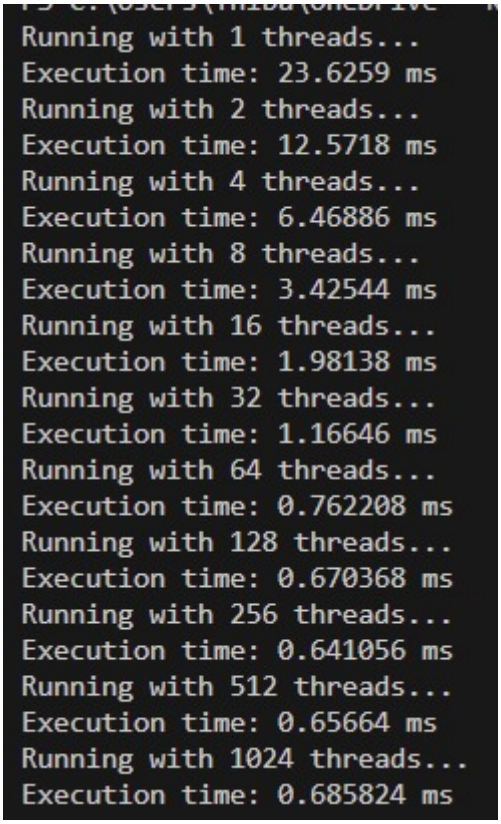


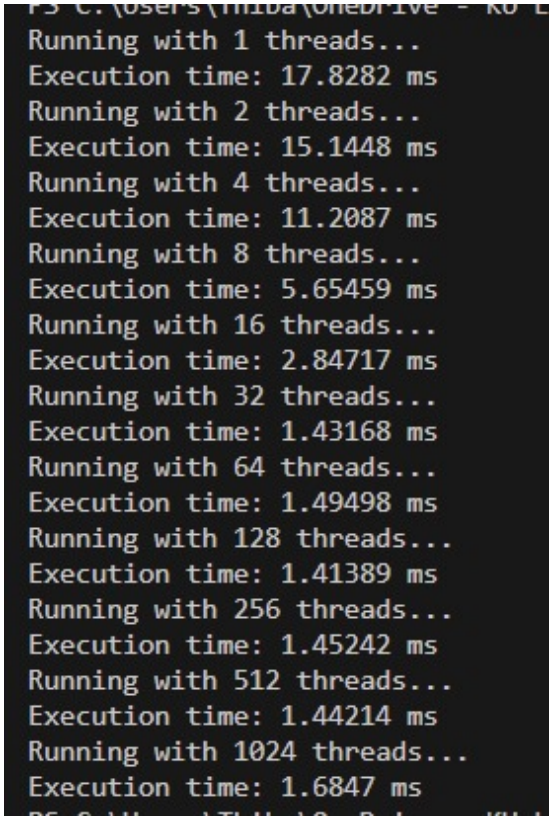
Figure 1: Influence Threads warp

2.2 Thread divergence

Since we only change the red value, there must be an if-statement. This means the program will take different execution branches within the warp, resulting in thread divergence.
In this part of the lab, we can see the execution times evolve similarly to in the first part which is logical. Warping and striding are still used, but an extra branch and a different calculation are causing the program to run slower on average. Also choosing how to process the pixels has an impact on execution time. Using two separate kernels will be faster than using one big kernel. This can once again be explained by the branching. When using two separate kernels there is no need for branching.



(a) Thread divergence using 2 separate kernels



(b) Thread divergence using 1 kernel

Figure 2: Comparison of thread divergence between using 2 kernels and 1 kernel

At last, we changed the processing of the pixels to happen in format RRR..RG..GB..B. Processing the pixels interleaved results in an improvement in execution time. We believe this is caused by having fewer memory transactions per warp.

```
Running with 1 threads...
Execution time: 17.4767 ms
Running with 2 threads...
Execution time: 8.20432 ms
Running with 4 threads...
Execution time: 4.10189 ms
Running with 8 threads...
Execution time: 2.0808 ms
Running with 16 threads...
Execution time: 1.06906 ms
Running with 32 threads...
Execution time: 0.580288 ms
Running with 64 threads...
Execution time: 0.512736 ms
Running with 128 threads...
Execution time: 0.491296 ms
Running with 256 threads...
Execution time: 0.504768 ms
Running with 512 threads...
Execution time: 0.511168 ms
Running with 1024 threads...
Execution time: 0.605184 ms
```

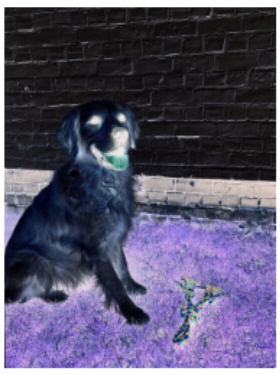
Figure 3: Interleaved

In the two tests, we observed that not using thread divergence results in faster execution time compared to using thread divergence. This is expected, as thread divergence introduces branching within a warp, causing some threads to stall while waiting for others to complete their execution, which means that the warp will take longer to complete.

2.3 Processed Images



(a) Gaussian transformation



(b) Inversion transformation

Figure 4: Results transformations

2.4 Code

2.4.1 Warps

```cpp
#include <cuda_runtime.h>
#include <iostream>
#include <cstdint>

#define M 960       // VR width
#define N 1280      // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

__global__ void invertImage(unsigned char *img, int width, int height, int channels) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height * channels;

    for (int i = idx; i < totalPixels; i += stride) {
        img[i] = 255 - img[i];
    }
}

void checkCudaError(cudaError_t err, const char *msg) {
    if (err != cudaSuccess) {
        std::cerr << msg << " (" << cudaGetErrorString(err) << ")\n";
        exit(EXIT_FAILURE);
    }
}

void invertImageCUDA(unsigned char *h_img, int width, int height, int channels, int numThreads) {
    int totalPixels = width * height * channels;
    int imgSize = totalPixels * sizeof(unsigned char);
    unsigned char *d_img;

    checkCudaError(cudaMalloc((void **)&d_img, imgSize), "Failed to allocate device memory");
    checkCudaError(cudaMemcpy(d_img, h_img, imgSize, cudaMemcpyHostToDevice), "Failed to copy image to device");

    int numBlocks = (totalPixels + numThreads - 1) / numThreads;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    invertImage<<<numBlocks, numThreads>>>(d_img, width, height, channels);
    cudaEventRecord(stop);

    checkCudaError(cudaGetLastError(), "Kernel execution failed");
    checkCudaError(cudaMemcpy(h_img, d_img, imgSize, cudaMemcpyDeviceToHost), "Failed to copy result to host");

    cudaEventSynchronize(stop);
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    std::cout << "Execution time: " << milliseconds << " ms\n";

    cudaFree(d_img);
}
```

```
54
55 uint8_t* get_image_array() {
56     FILE *imageFile = fopen("./anna.ppm", "rb");
57     if (imageFile == NULL) {
58         perror("ERROR: Cannot open input file");
59         exit(EXIT_FAILURE);
60     }
61
62     uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
63     fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET,
       imageFile);
64     fclose(imageFile);
65
66     return image_array + OFFSET;
67 }
68
69 void save_image_array(uint8_t* image_array) {
70     FILE *imageFile = fopen("./output_image.ppm", "wb");
71     if (imageFile == NULL) {
72         perror("ERROR: Cannot open output file");
73         exit(EXIT_FAILURE);
74     }
75
76     fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
77     fwrite(image_array, 1, M * N * C, imageFile);
78     fclose(imageFile);
79 }
80
81 int main() {
82     uint8_t* image_array = get_image_array();
83
84     for (int numThreads = 2; numThreads <= 1024; numThreads *= 2) {
85         std::cout << "Running with " << numThreads << " threads...\n";
86         invertImageCUDA(image_array, M, N, C, numThreads);
87     }
88
89     save_image_array(image_array);
90     free(image_array);
91     return 0;
92 }
```

2.4.2 Thread divergence

```
#include <cuda_runtime.h>
#include <iostream>
#include <cstdint>

#define M 960       // VR width
#define N 1280      // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

__global__ void processImage(unsigned char *img, int width, int height, int channels) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height * channels;

    for (int i = idx; i < totalPixels; i += stride) {
        int pixelIndex = i / channels;
        int colorIndex = i % channels;

        if (colorIndex == 0) { // Red channel: Apply Gaussian blur
            int left2 = (pixelIndex >= 2) ? img[i - 2 * channels] : 0;
            int left1 = (pixelIndex >= 1) ? img[i - channels] : 0;
            int right1 = (pixelIndex < totalPixels - channels) ? img[i + channels] : 0;
            int right2 = (pixelIndex < totalPixels - 2 * channels) ? img[i + 2 * channels] : 0;
            img[i] = (0.1 * left2 + 0.25 * left1 + 0.5 * img[i] + 0.25 * right1 + 0.1 * right2);
        } else { // Green & Blue: Invert colors
            img[i] = 255 - img[i];
        }
    }
}

void checkCudaError(cudaError_t err, const char *msg) {
    if (err != cudaSuccess) {
        std::cerr << msg << " (" << cudaGetErrorString(err) << ")\n";
        exit(EXIT_FAILURE);
    }
}

void processImageCUDA(unsigned char *h_img, int width, int height, int channels, int numThreads) {
    int totalPixels = width * height * channels;
    int imgSize = totalPixels * sizeof(unsigned char);
    unsigned char *d_img;

    checkCudaError(cudaMalloc((void **)&d_img, imgSize), "Failed to allocate device memory");
    checkCudaError(cudaMemcpy(d_img, h_img, imgSize, cudaMemcpyHostToDevice), "Failed to copy image to device");

    int numBlocks = (totalPixels + numThreads - 1) / numThreads;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    processImage<<<numBlocks, numThreads>>>(d_img, width, height, channels);
    cudaEventRecord(stop);
```

```
54
55      checkCudaError(cudaGetLastError(), "Kernel execution failed");
56      checkCudaError(cudaMemcpy(h_img, d_img, imgSize, cudaMemcpyDeviceToHost), "
        Failed to copy result to host");
57
58      cudaEventSynchronize(stop);
59      float milliseconds = 0;
60      cudaEventElapsedTime(&milliseconds, start, stop);
61      std::cout << "Execution time: " << milliseconds << " ms\n";
62
63      cudaFree(d_img);
64  }
65
66  uint8_t* get_image_array() {
67      FILE *imageFile = fopen("./anna.ppm", "rb");
68      if (imageFile == NULL) {
69          perror("ERROR: Cannot open input file");
70          exit(EXIT_FAILURE);
71      }
72
73      uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
74      fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET,
        imageFile);
75      fclose(imageFile);
76
77      return image_array + OFFSET;
78  }
79
80  void save_image_array(uint8_t* image_array) {
81      FILE *imageFile = fopen("./output_image.ppm", "wb");
82      if (imageFile == NULL) {
83          perror("ERROR: Cannot open output file");
84          exit(EXIT_FAILURE);
85      }
86
87      fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
88      fwrite(image_array, 1, M * N * C, imageFile);
89      fclose(imageFile);
90  }
91
92  int main() {
93      uint8_t* image_array = get_image_array();
94
95      for (int numThreads = 16; numThreads <= 1024; numThreads *= 2) {
96          std::cout << "Running with " << numThreads << " threads...\n";
97          processImageCUDA(image_array, M, N, C, numThreads);
98      }
99
100     save_image_array(image_array);
101     free(image_array);
102     return 0;
103 }
```

2.4.3 Thread divergence with 2 kernels

```
#include <cuda_runtime.h>
#include <iostream>
#include <cstdint>

#define M 960       // VR width
#define N 1280      // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

#define BLOCK_SIZE 32

// Kernel red color
__global__ void processRedChannel(unsigned char *img, int width, int height, int channels) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height * channels;

    for (int i = idx * channels; i < totalPixels; i += stride * channels) {
        int pixelIndex = i / channels;
        int left2 = (pixelIndex >= 2) ? img[i - 2 * channels] : 0;
        int left1 = (pixelIndex >= 1) ? img[i - channels] : 0;
        int right1 = (pixelIndex < totalPixels - channels) ? img[i + channels] : 0;
        int right2 = (pixelIndex < totalPixels - 2 * channels) ? img[i + 2 * channels] : 0;
        img[i] = (0.1 * left2 + 0.25 * left1 + 0.5 * img[i] + 0.25 * right1 + 0.1 * right2);
    }
}

// Kernel green and blue
__global__ void processImage(unsigned char *img, int width, int height, int channels) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height * channels;

    for (int i = idx; i < totalPixels; i += stride) {
        int colorIndex = i % channels;
        if (colorIndex != 0) {
            img[i] = 255 - img[i];
        }
    }
}

void checkCudaError(cudaError_t err, const char *msg) {
    if (err != cudaSuccess) {
        std::cerr << msg << " (" << cudaGetErrorString(err) << ")\n";
        exit(EXIT_FAILURE);
    }
}

void processImageCUDA(unsigned char *h_img, int width, int height, int channels, int numThreads) {
    int totalPixels = width * height * channels;
    int imgSize = totalPixels * sizeof(unsigned char);
    unsigned char *d_img;

    checkCudaError(cudaMalloc((void **)&d_img, imgSize), "Failed to allocate
```

```
    device memory");
    checkCudaError(cudaMemcpy(d_img, h_img, imgSize, cudaMemcpyHostToDevice), "
    Failed to copy image to device");

    int numBlocks = (totalPixels + numThreads - 1) / numThreads;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);
    processRedChannel<<<numBlocks, BLOCK_SIZE>>>(d_img, width, height, channels);
    processImage<<<numBlocks, numThreads>>>(d_img, width, height, channels);
    cudaEventRecord(stop);
    checkCudaError(cudaGetLastError(), "Kernel execution failed");
    checkCudaError(cudaMemcpy(h_img, d_img, imgSize, cudaMemcpyDeviceToHost), "
    Failed to copy result to host");

    cudaEventSynchronize(stop);
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    std::cout << "Execution time: " << milliseconds << " ms\n";

    cudaFree(d_img);
}

uint8_t* get_image_array() {
    FILE *imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET,
    imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

int main() {
    uint8_t* image_array = get_image_array();
    for (int numThreads = 1; numThreads <= 1024; numThreads *= 2) {
        std::cout << "Running with " << numThreads << " threads...\n";
        processImageCUDA(image_array, M, N, C, numThreads);
    }
    save_image_array(image_array);
    free(image_array);
    return 0;
}
```

2.4.4 Thread divergence with interleaved image

```
#include <cuda_runtime.h>
#include <iostream>
#include <cstdint>
#include <cstring>

#define M 960       // VR width
#define N 1280      // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

#define BLOCK_SIZE 32

void convertToPlanar(unsigned char *input, unsigned char *output, int width, int height) {
    int totalPixels = width * height;
    int redOffset = 0;
    int greenOffset = totalPixels;
    int blueOffset = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[redOffset + i] = input[i * C];       // red
        output[greenOffset + i] = input[i * C + 1]; // green
        output[blueOffset + i] = input[i * C + 2];  // blue
    }
}

void convertToInterleaved(unsigned char *input, unsigned char *output, int width, int height) {
    int totalPixels = width * height;
    int redOffset = 0;
    int greenOffset = totalPixels;
    int blueOffset = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[i * C] = input[redOffset + i];       // red
        output[i * C + 1] = input[greenOffset + i]; // green
        output[i * C + 2] = input[blueOffset + i];  // blue
    }
}

// Kernel red color
__global__ void processRedChannelPlanar(unsigned char *img, int width, int height) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int totalPixels = width * height;
    int redOffset = 0;

    for (int i = idx; i < totalPixels; i += stride) {
        int left2 = (i >= 2) ? img[redOffset + i - 2] : 0;
        int left1 = (i >= 1) ? img[redOffset + i - 1] : 0;
        int right1 = (i < totalPixels - 1) ? img[redOffset + i + 1] : 0;
        int right2 = (i < totalPixels - 2) ? img[redOffset + i + 2] : 0;
        img[redOffset + i] = (0.1 * left2 + 0.25 * left1 + 0.5 * img[redOffset + i] + 0.25 * right1 + 0.1 * right2);
    }
}

// Kernel green and blue
__global__ void processImagePlanar(unsigned char *img, int width, int height) {
```

```
57      int idx = blockIdx.x * blockDim.x + threadIdx.x;
58      int stride = gridDim.x * blockDim.x;
59      int totalPixels = width * height;
60      int greenOffset = totalPixels;
61      int blueOffset = 2 * totalPixels;
62
63      for (int i = idx; i < totalPixels; i += stride) {
64          img[greenOffset + i] = 255 - img[greenOffset + i];
65          img[blueOffset + i] = 255 - img[blueOffset + i];
66      }
67  }
68
69  void checkCudaError(cudaError_t err, const char *msg) {
70      if (err != cudaSuccess) {
71          std::cerr << msg << " (" << cudaGetErrorString(err) << ")\n";
72          exit(EXIT_FAILURE);
73      }
74  }
75
76  void processImageCUDA(unsigned char *h_img, int width, int height) {
77      int totalPixels = width * height * C;
78      int imgSize = totalPixels * sizeof(unsigned char);
79      unsigned char *d_img;
80
81      checkCudaError(cudaMalloc((void **)&d_img, imgSize), "Failed to allocate
        device memory");
82      checkCudaError(cudaMemcpy(d_img, h_img, imgSize, cudaMemcpyHostToDevice), "
        Failed to copy image to device");
83
84      int numBlocks = (totalPixels / C + BLOCK_SIZE - 1) / BLOCK_SIZE;
85
86      cudaEvent_t start, stop;
87      cudaEventCreate(&start);
88      cudaEventCreate(&stop);
89
90      cudaEventRecord(start);
91      processRedChannelPlanar<<<numBlocks, BLOCK_SIZE>>>(d_img, width, height);
92      processImagePlanar<<<numBlocks, BLOCK_SIZE>>>(d_img, width, height);
93      cudaEventRecord(stop);
94
95      checkCudaError(cudaGetLastError(), "Kernel execution failed");
96      checkCudaError(cudaMemcpy(h_img, d_img, imgSize, cudaMemcpyDeviceToHost), "
        Failed to copy result to host");
97
98      cudaEventSynchronize(stop);
99      float milliseconds = 0;
100     cudaEventElapsedTime(&milliseconds, start, stop);
101     std::cout << "Execution time: " << milliseconds << " ms\n";
102
103     cudaFree(d_img);
104 }
105
106 int main() {
107     uint8_t* interleaved_image = (uint8_t*)malloc(M * N * C * sizeof(uint8_t));
108     uint8_t* planar_image = (uint8_t*)malloc(M * N * C * sizeof(uint8_t));
109
110     // reading is interleaved (could've been supplied as planar mate..)
111     FILE *imageFile = fopen("./anna.ppm", "rb");
112     fread(interleaved_image, sizeof(uint8_t), M * N * C, imageFile);
113     fclose(imageFile);
114
115     convertToPlanar(interleaved_image, planar_image, M, N);
```

```
116
117     processImageCUDA(planar_image, M, N);
118
119     convertToInterleaved(planar_image, interleaved_image, M, N);
120
121     FILE *outputFile = fopen("./output_image.ppm", "wb");
122     fprintf(outputFile, "P6\n%d %d\n255\n", M, N);
123     fwrite(interleaved_image, 1, M * N * C, outputFile);
124     fclose(outputFile);
125
126     free(interleaved_image);
127     free(planar_image);
128     return 0;
129 }
```